明镜 · 开工会材料（给 IT 团队）

> **这份给谁**：**刚组建的技术团队。** **干什么用**：**照着走**——不是"查的"，是"照着走的"。所以它和 `3_技术方案.md` 的分工是：| 文件 | 用法 ||---|---|| `3_技术方案.md`（含附录 A 表结构 / 附录 B 接口清单） | **查**：遇到具体问题翻它 || **本文件** | **走**：第一天做什么、第一周验收什么 | **验收标准**：**给一个刚组建的团队，他们照着能开工，不用再来问"第一步干什么"。** 2026 年 10 月 · 陕西驭界科技有限公司

零、这份材料的核心是一句话

> **# 顺序比内容重要。先做错顺序，后面全返工。**

这不是管理偏好，是技术耦合**决定的**：

```
数据模型（地基）
      ↓ 决定
接口形状（上层）
      ↓ 决定
页面（最上层）
```

改动成本的规律：

改动	**代价**
改页面	改几个文件，一天
改接口	前后端都要改，几天
改数据模型（身份、租户、权限、计量）	**所有上层的代码全部返工**

> **所以"地基四件"必须先做——它不是因为"重要"，是因为"改起来最贵"。**

一、开工前必须确认的三件事（不做完不许开工）

① 团队配置（谁干什么）

角色	**人数**	**第一周具体负责什么**
后端	2	**"地基四件"**（账号/租户/权限/计量）

前端	1-2	**用 Mock 数据先把 5 个骨架页面搭出来**（不等后端）
AI 工程	1	计量与分级模型的接入骨架（**第一周只需要把"调用要记三个标记"这件事做成规范**）
运维/安全	0.5	环境、CI、备份（**那两条 CI 测试是他第一周的核心产出**）

★ **一条纪律**：**前端不等后端。 接口约定（附录 B）已经定好了路径和动作**——前端按它 mock 数据先做页面，**不要等接口写完。 "等接口"是项目里最贵的等待**（两边都闲着）。

---

② 环境与工具（开工前要有的）

项	要求	为什么
代码仓库	一个仓库（单体，**一期不做微服务**）	团队小、负载低；**微服务会把开发速度拖慢一半**
数据库	关系型 + **支持行级安全（RLS）**	★ **多租户双保险的第二道就靠它**——选型时这条是硬指标
CI	每次提交自动跑测试	**那两条 CI 测试（越权、跨租户）必须跑在 CI 里**，不是"有空手动跑一次"
按环境分离	开发 / 测试 / 生产	生产库里绝对不能有假数据

③ 谁定什么（避免"每件事都要开会"）

事项	谁定	到什么程度
技术选型（语言/框架/中间件/开源底座）	**IT 负责人**（你们定，我不定）	只要满足 3_技术方案 9.1 的**五条约束**
表结构的字段类型、索引写法	后端	守住**附录 A 的十条通用约定 + 八条"不可能"**
接口的入参出参	后端 + 前端共同	守住**附录 B 的命名规范与四类前缀**
产品需求的理解	**先看 `1_产品方案.md`**；还是不清楚 → 找产品（我）	**不要靠猜需求开工**
优先级冲突	找产品（我）	判据：**这条不做，"1,000 家公会、10 万用户"会不会受影响**

★ **一句话：技术怎么实现，你们定；"什么东西不能破"，我定。** 那两条界线分别在 3_技术方案 9.1（五条约束）和 10.2（十条自检）。

二、第 1 天：两件事，只做两件

为什么第 1 天只做两件：这两件是**"整个方案能不能成立"的探针——它们要是一天做不出来，说明架构选错了，越早发现越好。**

第 1 件：跑通"一套代码出手机和电脑"

项	内容
做什么	搭一个空项目，写**一个页面**（比如登录页的静态版），**在手机浏览器和电脑浏览器上各打开一次**
验收标准	手机上不用横向滚动、字看得清；电脑上**同一个页面**只是更宽（**组件尺寸不变**）
为什么第 1 天就做	**这是五条约束里的第 1 条。**如果选的技术栈做不到"一套代码两套屏"，**要改的是整个前端方案**——越晚发现越贵
反例	✕做两个前端工程（手机一套、电脑一套）→ 维护成本翻倍、两边不一致

注意：**"更宽"不是"另一套布局"。** 所有组件（按钮高度、字号）在电脑上不变，**变的只有"一屏能放多少"**（设计规范 4.3）。

第 2 件：跑通"登录 → 身份 → 会话"这条链路

项	内容
做什么	从登录开始，走通：**账号 → 身份 → 发 token → 请求带上身份 → 数据自动收窄到"我的公司"**
验收标准	写**一条测试**：**故意不写公司条件去查，断言返回 0 条**（不是"返回了别家数据"）
为什么第 1 天就做	这是**"四道门"和"多租户"的共同地基。**它跑通了，后面所有接口都是"天生安全"；它跑不通或晚做，**后面要一条条补**
反例	✕先做业务接口，等有空再加租户过滤 → **等于写完 64 个接口再回来检查每个都可能漏**

★ 这一天最有价值的产出不是代码，是那条测试。 它证明"开发者忘了写，数据也漏不出去"——这句话是尽调时说的，不是开发时说的。

三、第 1 周：只做"地基四件"

这一周不许开任何业务接口。 理由见第零节：地基是"改起来最贵"的那一层。

第 1 件：账号 / 身份 / 任职 三张表 + 统一登录网关

项	内容
建什么	`user` / `identity` / `employment`（附录 A.1）
实现什么	登录、签发 token、**身份切换**（一个人多个身份）、请求带身份上下文
验收标准 1	**造一个"运营兼主播"的账号**：他能在登录后**选进哪道门**，而且两个身份的数据互不干扰
验收标准 2	新增一个公会成员，**只改数据、不改代码**（岗位和职级是数据，不是代码）

★ 为什么"运营兼主播"这条是第一个验收标准： 现实里最常见的一种人（既是某公会的运营，自己也在播）。 **如果身份模型写死"一个账号一个类型"，第一个真实案例就会撞上**—— 那时改的代价是：**所有已写的接口的权限判断全部要重看一遍。**

第 2 件：多租户中间件 + 行级安全 + 那条 CI 测试

项	内容
实现什么	① 数据访问中间件**自动**注入"公司 = 我的公司" ② 数据库**行级安全**再查一遍 ③ **CI 测试**：故意漏写条件，断言返回 0 条
验收标准	**在 CI 里能看到这条测试跑过**（不是本地手动跑过一次）

★ 这条测试要进 CI，不是"有空跑"。 双保险的价值在于"开发者犯错时也不出事"——没有自动化验证，它就只是"看起来有两道"。

第 3 件：权限三层（含字段标签）

项	内容

实现什么	① 功能权限（角色 × 权限码）② 范围规则（**本人/本组/本部门/全公司**，只有 4 条）③ **字段标签**（公开/范围内/点名）
怎么落	**权限不在业务代码里判，在"数据出口"统一做**（技术方案 3.1）
验收标准 1	**改一个字段的可见岗位，不需要改代码、不需要发版**
验收标准 2	用低权限账号问 AI"公司里谁工资最高"，**AI 答不出来**（AI 的权限 = 提问者的权限）

★ 验收标准 2 容易被漏掉，但它是最重要的： AI 是最容易变成"权限后门"的地方—— 整套权限做完，被一个 AI 接口绕过去，那是白做。

第 4 件：AI 计量表 + 三个标记的强制规范

项	内容
建什么	`ai_usage` 表（附录 A.4），**按月分区**
规范什么	**★ 任何调用 AI 的代码，必须带三个标记：谁调的 / 哪家公司 / 哪个功能。缺一个，代码评审不通过。**
验收标准 1	`ai_usage` 表里**不存在任何一条缺标记的记录**
验收标准 2	能按**四个粒度**查出来：全平台 / 每家公司 / 每个人 / **每一类功能**

★ 为什么这一件必须第 1 周做（而不是"等有 AI 功能了再加"）： ① **事后补不了**——历史数据不存在了； ② **内测期的数据就是单位经济模型的第一版输入——内测没记账，等于白内测。 这件事直接决定"800 万撑 6 个月还是 18 个月"**（成本三档差 3-13 倍）。

第 1 周末的验收清单（★ 逐条答"是/否"，不许"大概是"）

#	自检	判定
1	**一个人能有两个身份**，且能选门进入	☐
2	**故意不写公司条件，查询返回 0 条**	☐
3	这条测试**在 CI 里跑**（不是手动跑过）	☐
4	改字段可见性**不用改代码、不用发版**	☐
5	低权限账号问 AI 敏感问题，**AI 答**	☐

	不出	
6	`ai_usage` **没有一条缺三个标记的记录**	☐
7	成本能按**四个粒度**查	☐

> **七条里任何一条答"否"，这一周就没有结束。 因为它们是"后面 64 个接口能不能天生安全"的前提。**

四、第 2 周：地基之上，先做"门和入驻"

> **为什么第 2 周做这个而不是做公会业务： 没有"门"，公会进不来；没有"入驻"，1,000 家是空话。**
> 而这两块**都建立在第 1 周的地基上**（所以顺序不能换）。

第 2 周上半：`/pub/*` + `/platform/*` 的骨架

做什么	验收标准
登录 / 验证码 / **身份切换**（`/pub`）	一个人两个身份，能切
入驻申请提交 + 审核 + 一键开号（`/platform`）	**一家公会能从"填表"走到"开号"**，且开号时**自动装入行业默认岗位模板**
激活漏斗的**数据来源**（先有数据，页面后做）	能算出"申请 → 开户 → 激活"各步的转化

> **★ "一键开号 + 装入行业默认模板"是这一段的核心—— 产品方案 4.3 写过：新公会开号看到空系统，80% 的人在这里就走了。 所以"开号"不是"建一条记录"，是"建一套能立刻用的东西"。**

第 2 周下半：公会端的"组织与权限"（`/org` 前 6 个接口）

做什么	验收标准
部门树、员工列表/详情、**改岗位职级**、岗位列表、**改字段可见性**	**客户自己能在界面上配岗位和字段，不用找我们**

> **★ 这一段的验收标准看着简单，但它是"加一家客户不用发版"的落点—— 没有它，每来一家公会我们都要改一次代码，2-3 个商务铺 1,000 家根本不可能。**

第 2 周末的里程碑：一家公会能被完整地"开出来"

申请（5 分钟填完）→ 审核（点两个按钮）→ 开号（自动装行业模板）
→ 管理员登录 → 看到的是"装好的系统"，不是空白

这个里程碑一旦跑通，"能不能铺 1,000 家"这个问题就从"想法"变成了"能演示的东西"。

五、前两周的里程碑一览

时间	应该能跑起来什么	这一天验证了什么
第 1 天	**一套代码两套屏 + 登录链路 + 那条"不写条件返回 0 条"的测试**	**架构选型对不对**
第 3 天	账号/身份/任职三张表 + 登录网关；**"运营兼主播"能选门**	**身份模型对不对**
第 5 天	多租户双保险 + **CI 里的越权/跨租户测试**	**数据隔离真的双**
第 7 天	权限三层（含字段标签，客户可改）	**权限绕不过去**
第 8 天	`ai_usage` 计量 + 三个标记规范 + 四个粒度查询	**成本算得出来**
第 10 天	`/pub` 登录链路 + **身份切换**	**四道门在代码里成立**
第 12 天	`/platform` 入驻申请 + 审核 + **一键开号装模板**	**公会进得来**
第 14 天	`/org` 组织与权限 6 个接口 + 客户自助配岗位/字段	**加客户不用发版**

> **★ 这张表的作用不是"考核进度"，是"每天问一句：今天验证了哪个假设"。 假设错得越早发现越便宜。**

六、前两周不做什么（连同理由）

✕不做	为什么
任何业务接口（跟播、复盘、主播档案…）	**地基没验收完就开业务，等于在沙子上盖楼**
微服务	团队小、负载低（很多人做很少的事）；**微服务会把开发速度拖慢一半**
完整 UI（照着原型精修）	第 1-2 周**只要"能跑通链路"**——原型是给"看明白"用的，不是给"精修"用的
多平台对接（企微 / 钉钉）	第一阶段只做飞书；**为了"支持三个平台"把第一版做复杂是自伤**
自建向量检索	知识库先用关键词检索，**等它真的大了再说**

App / 小程序	一期只发网页（**"下载安装"是铺用户阶段最大的漏斗**）
自研模型 / 微调	成本极高，且**行业数据还没攒够**

七、第 3 周起：按一期清单铺业务（顺序建议）

第 3 周　主播与七段流水线（`/org/anchor/*` 8 个接口）
★ 先做「主播档案 · 一页看完」那个聚合接口（原型 `/g/anchor`）
理由：它把 11 个交接点收在一个人身上，是整条业务线的"中心"

第 4 周　排班与跟播（含 ★ 5 字段的录入 + 提交时核对）+ AI 对话
★ 跟播记录是"数据真假的开关"—它跑通了，"数据壁垒"这个故事才成立

第 5 周　自动复盘（读已有的妙记录音）+ 日报周报
★ 明镜最值钱的一个功能

第 6 周　招聘面试 / 培训 / 形象与视觉 / 内容
（按一期清单里标"　二期"的项，视排期插入）

第 7-8 周　主播门 `/me/*`（9 个接口）
★ 门和账号体系第 1 周就该留好，这里只是"把门后的内容做出来"

★ **一条提醒：这个顺序不是"功能多少"排的，是"依赖关系"排的。** 比如"跟播记录"排在"自动复盘"前面——**因为复盘读的是跟播记录 + 妙记录音，它依赖前者。**

八、遇到问题找谁

问题类型	找谁	响应方式
技术选型、架构、实现方式	**IT 负责人（你们自己定）**	只要不破 3_技术方案 9.1 的五条约束
需求不清楚（这个功能到底要解决什么）	**先看 `1_产品方案.md`**	那份里每一节都写了**"影响什么商业结果"**
需求冲突 / 优先级	产品（我）	判据：**这条不做，"1,000 家公会、10 万用户"会不会受影响**
界面细节（组件、字号、间距）	看 2_开发原型.html + 设计规范	原型里每一页都标了**"为什么这么设计"**
"这样做行不行"（拿不准会不会破规矩）	对 3_技术方案 第十节的**十条自检**	**十条里能答"是"就可以做**
要改已经定下来的东西	产品（我）	**改完要过 `4_收口清单` 的三道查法**（说法一致 / 说了没做 / 做了没说）

★ **最后一条要特别说**： **"定稿"不等于"不能改"，等于"改之前先对账"。** 你们在实现中发现方案有问题，**直接提**——但提的时候请说清**这会影响三份文件里的哪几处。**

九、开工会的最后一页：三句话

① **顺序比内容重要。** 地基四件（账号身份 / 多租户 / 权限 / 计量）先做完—— **不是因为它重要，是因为它改起来最贵。** ② **那两条测试比功能重要。** "不写公司条件返回 0 条"、"用主播身份调公会接口全被拒"—— **这两条测试，是"能给人看"和"不能给人看"的分界线。** ③ **遇到拿不准的，回到一句话： 这条不做，"1,000 家公会、10 万用户"会不会受影响？ 会 → 做。不会 → 往后放。**

配套文件：`3_技术方案.md`（含附录 A 表结构 / 附录 B 接口清单）· `1_产品方案.md` · `2_开发原型.html` · `4_收口清单.md`